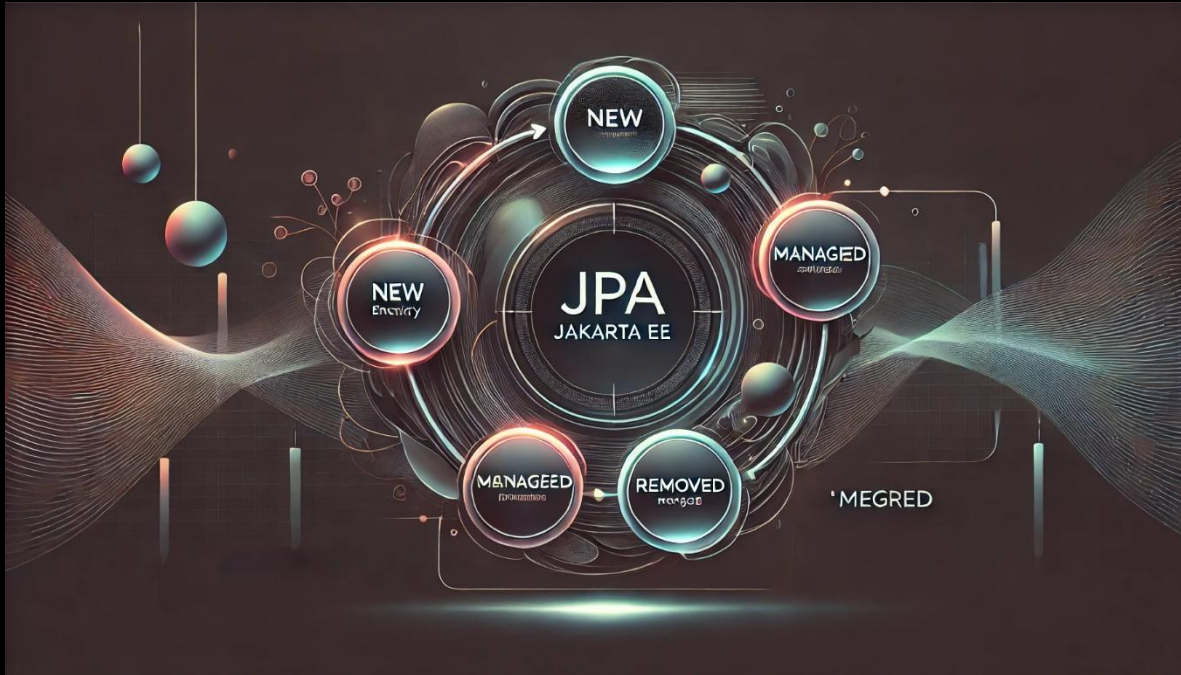


Recuperar un objeto con JPA y Jakarta EE



Recuperar un objeto con JPA y Jakarta EE

✓ 1. Crear la clase para encontrar un objeto con JPA

Abrimos **NetBeans** y en nuestro proyecto **holamundo-jpa**, dentro del paquete `sga.cliente.ciclodevida`, creamos una nueva clase llamada `EncontrarObjetoJPA`.

✦ Pasos:

1. En **NetBeans**, abrimos el **paquete** `sga.cliente.ciclodevida`.
2. Hacemos **clic derecho** en el paquete y seleccionamos:
New > Java Class
3. Nombramos la clase como `EncontrarObjetoJPA` y damos **Finalizar**.

✓ 2. Agregar la configuración inicial

Ahora, agregamos el código base para la conexión con JPA.

✦ Código inicial de la clase `EncontrarObjetoJPA`:

```
package sga.cliente.ciclodevida;  
  
import jakarta.persistence.EntityManager;
```

```
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.EntityTransaction;
import jakarta.persistence.Persistence;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class EncontrarObjetoJPA {

    private static final Logger log = LoggerFactory.getLogger(EncontrarObjetoJPA.class);

    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
        //Inicia la transaccion
        //Paso 1. Iniciar una transaccion
        try (EntityManager em = emf.createEntityManager()) {
            //Inicia la transaccion
            //Paso 1. Iniciar una transaccion
            EntityTransaction tx = em.getTransaction();
            tx.begin();
            //Paso 2. Ejecuta SQL de tipo select
            Persona persona1 = em.find(Persona.class, 10);
            //Paso 3. termina la transaccion
            tx.commit();
            //Objeto en estado de detached
            log.info("Objeto recuperado en estado de detached: " + persona1);
            //Cerramos el entity manager
        }
    }
}
```

✖ Explicación del código:

- **EntityManagerFactory emf:** Crea una conexión con la unidad de persistencia (SgaPU definida en persistence.xml).
- **EntityManager em:** Se usa para interactuar con la base de datos.
- **EntityTransaction tx:** Maneja la transacción.
- **tx.begin():** Inicia la transacción.
- **em.find(Persona.class, 1):** Busca una persona en la base de datos con id=1.
- **tx.commit():** Confirma la transacción.
- **log.debug():** Imprime el objeto encontrado.
- **em.close():** Cierra la conexión. Si se usó try-with-resources se cierra el recurso de manera automática.

✓ 4. Ejecutar la búsqueda de objetos con JPA

Ahora ejecutamos la clase `EncontrarObjetoJPA`:

✚ Pasos:

1. Abrimos la base de datos en MySQL Workbench

- Ejecutamos:

```
SELECT * FROM persona;
```

- Aseguramos que existe una persona con `id=1`

2. Ejecutamos el código en NetBeans:

- Clic derecho en `EncontrarObjetoJPA.java` → Run File.
-

✓ 5. Validar el resultado en consola

✚ Resultados esperados en la consola:

Si el ID existe:

```
DEBUG - Objeto recuperado en estado detached: Persona{idPersona=1,
nombre='Juan', apellido='Pérez', email='jperez@mail.com',
telefono='5551234567'}
```

Si el ID no existe:

```
DEBUG - Objeto recuperado en estado detached: null
```

✓ 6. Probar con diferentes IDs

- Cambia `em.find(Persona.class, 1)`; por otro ID válido en la base de datos.
 - Si el ID **no existe**, el resultado será `null`.
-

✓ Conclusión

Hemos aprendido a:

- ✓ Configurar **persistence.xml** para conexión local.
- ✓ Buscar un objeto en **JPA** con `em.find()`.
- ✓ Manejar correctamente **transacciones en JPA**.
- ✓ Imprimir el objeto encontrado en la consola.

 **Con esto ya podemos encontrar objetos en la base de datos con JPA!**

Sigue adelante con tu aprendizaje  , ¡el esfuerzo vale la pena!

¡Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)